

Engineering Plan: Understanding Ghostty's Codebase for tmux Control Mode Integration

Revision: 0 (initial draft)

Goal

Help a junior engineer understand all parts of the Ghostty codebase that will be touched by a Ghostty plus tmux control mode integration. The engineer is new to Zig and has never worked on terminals or terminal emulators.

Part A: Three Learning Approaches

Approach 1: Top-Down Reading ("Understand the Architecture First")

When to choose this approach: Choose this when you are the kind of learner who needs to see the big picture before any details make sense. If you find yourself asking "but why does this exist?" when reading unfamiliar code, start here. This approach is also the right choice if you are going to be making architectural decisions (like designing the new tmux backend) rather than implementing within an existing design.

Why it works: The tmux control mode integration spans five layers of the Ghostty codebase: protocol parsing, semantic state management, integration glue, terminal I/O, and GUI. Reading top-down gives you a mental map of how data flows through these layers before you get lost in the details of any one layer. You will know what each file is for and why it exists before you read a single line of its implementation.

Milestone 1: Understand the product and protocol

Steps:

1. Read the tmux wiki page on Control Mode at github.com/tmux/tmux/wiki/Control-Mode. Read it completely, not skimming. Pay attention to the list of notification types, the escaping rules, and the flow control mechanism.
2. Run `tmux minus-C` (single C) new-session minus-s test in any terminal. This gives you human-readable control mode. Type the command `list-windows` and press Enter. Observe the percent-begin, the output line, and the percent-end. Type `list-panes` and observe the output. Type `display-message minus-p hash-open-brace version close-brace` and observe.
3. In another terminal window, attach to the same session with `tmux attach minus-t` test. Type some commands. Go back to the control mode terminal and observe the percent-output lines that appeared.
4. In the attached session, run `tmux split-window`. Go back to the control mode terminal and observe the percent-layout-change notification.
5. Write down, in your own words, a list of the notification types you observed and what each one means.

Success criterion: You can explain to another engineer what tmux control mode is, what the protocol looks like, and how a terminal emulator would use it to create native tabs and splits. You can describe the

difference between percent-output (live pane data), percent-begin/percent-end blocks (command responses), and async notifications (percent-layout-change, percent-window-add, etc.).

Milestone 2: Understand the data flow through Ghostty

Steps:

1. Read the diagrams.md file in the repository root, focusing on the "Data flow: DCS bytes to screen pixels" diagram. Trace the path from tmux server through dcs.zig, control.zig, stream_handler.zig, viewer.zig, and the TODO gap.
2. Read src/terminal/tmux.zig (the module re-export file, about 14 lines). Note which types are exported and from which files.
3. Read src/terminal/main.zig line 26 to see the conditional export. Understand that the tmux module only exists when tmux_control_mode is true.
4. Read src/terminal/build_options.zig line 46 to see that tmux_control_mode is tied to Oniguruma availability.
5. Draw your own diagram (on paper or whiteboard) showing how bytes flow from the tmux process to the Viewer, labeling each file and the data type passed between them.

Success criterion: You can draw, from memory, the pipeline from tmux bytes to the Viewer's windows action, naming each file and the type that crosses each boundary (bytes to dcs.zig, Notification to stream_handler.zig, Action from viewer.zig). You can point to the exact location where the pipeline breaks (stream_handler.zig, the windows action TODO).

Milestone 3: Read each file's public interface

Steps:

1. Open src/terminal/tmux/control.zig. Read only the Notification union (near the end of the file, around line 498) and the Parser struct definition (near the top). Do not read the implementation of put or parseNotification yet. Write down what notification types exist and what each one carries.
2. Open src/terminal/tmux/layout.zig. Read only the Layout struct definition and its Content union. Write down the three possible content types (pane, horizontal, vertical) and what fields each has.
3. Open src/terminal/tmux/output.zig. Read only the Variable enum and the FormatStruct function signature. Write down what kind of tmux state the output parser can extract.
4. Open src/terminal/tmux/viewer.zig. Read the State enum, the Action union, the Window struct, and the Pane struct. Read the ASCII lifecycle diagram between lines 55 and 144 carefully. Write down the four states and what transitions between them.
5. Open src/termio/stream_handler.zig. Read lines 389 through 461 (the tmux DCS command handling). Write down what happens for each notification type and each action type.
6. Open src/termio/backend.zig. Read the Kind enum and Backend union. Note that there is only exec today.

Success criterion: You have a written summary (your own notes, not copied) of each file's public interface: what types it defines, what they represent, and how they connect to the types in adjacent files. You have identified the exact line where the windows action is dropped.

Milestone 4: Read implementations with purpose

Steps:

1. In `control.zig`, read the `put` method (the main parsing loop). Trace the state transitions: `idle` receives a percent byte and transitions to `notification`; `notification` accumulates bytes until a newline and calls `parseNotification`; `parseNotification` dispatches based on the notification name; `percent-begin` transitions to `block` state; `block` accumulates lines until a valid guard line.
2. In `viewer.zig`, read the `next` method (main dispatch), then `nextStartupBlock`, `nextStartupSession`, and `nextCommand`. For `nextCommand`, trace what happens when each command type's response arrives: `tmux_version` stores the version, `list_windows` triggers `syncLayouts`, `pane_history` feeds content to a `Terminal`, and so on.
3. In `viewer.zig`, read `syncLayouts` (how the `Viewer` reconciles its internal pane map with a new layout tree) and `initLayout` (how it creates `Terminal` instances for new panes and queues capture commands).
4. Read the tests in `viewer.zig` starting around line 1496. These are the best documentation of expected behavior. Read at least the "initial flow" test and the "two pane flow with pane state" test completely.

Success criterion: You can explain, step by step, what happens from the moment `Ghostty` detects `ESC P 1000 p` to the moment the `Viewer` has a fully initialized `Terminal` for each pane, including every command sent to `tmux` and every response processed. You can explain what `syncLayouts` does when a layout change adds a new pane.

Approach 2: Bottom-Up Instrumentation ("Run It and Watch What Happens")

When to choose this approach: Choose this when you learn best by doing, not by reading. If abstract explanations do not stick until you have seen the concrete behavior, start here. This approach is particularly good if you have access to a `Ghostty` build environment and can compile and run `Ghostty` locally.

Why it works: The `tmux` control mode pipeline already works internally — the parser parses, the `Viewer` discovers windows and panes, the command queue sends and receives commands. You just cannot see any of it from the GUI because the windows action is dropped. By adding logging and running experiments, you can observe the entire pipeline in action and build your understanding from concrete observations.

Milestone 1: Set up the development environment

Steps:

1. Clone the `Ghostty` repository and build it from source. Follow the build instructions in the repository's documentation. You need a working `Ghostty` binary that you can run from the command line.
2. Verify your build by running the `Ghostty` binary from a terminal. A `Ghostty` window should open. Type a command in it to verify it works as a normal terminal.
3. Install `tmux` if it is not already installed. On macOS, use `brew install tmux`. Verify by running `tmux minus-V` to see the version.
4. Verify the test suite works by running `zig build test minus-Dtest-filter equals tmux` from the repository root. All `tmux`-related tests should pass.

Success criterion: You have a working `Ghostty` build, `tmux` is installed, and `zig build test minus-Dtest-filter equals tmux` passes.

Milestone 2: Observe the control mode protocol directly

Steps:

1. In any terminal, create a detached tmux session: `tmux new-session minus-d minus-s experiment`.
2. In the same terminal, start control mode with echo: `tmux minus-C attach minus-t experiment`. You will see protocol messages printed as plain text.
3. Observe the initial output: a percent-begin/percent-end block (the startup block) and a percent-session-changed notification.
4. Type `list-windows` and press Enter. Observe the percent-begin line, the output line (containing session ID, window ID, dimensions, and layout string), and the percent-end line.
5. Open a second terminal and attach to the same session normally: `tmux attach minus-t experiment`. In that terminal, type `echo hello`. Go back to the control mode terminal and observe the percent-output line.
6. In the attached session, run `tmux split-window`. Go back to the control mode terminal and observe the percent-layout-change notification. Note how the layout string changed to include a second pane.
7. In the attached session, run `tmux new-window`. Observe the percent-window-add notification in the control mode terminal.

Success criterion: You have seen every major notification type in the raw protocol. You can correlate tmux operations (split, new window, typing) to specific protocol messages.

Milestone 3: Observe Ghostty's internal processing**Steps:**

1. Open two terminal windows (these can be in any terminal, not necessarily Ghostty).
2. In Terminal A, launch Ghostty from the command line with log output visible. The exact command depends on your platform, but something like: `path-to-ghostty 2 greater-than ampersand 1 pipe grep minus-i tmux`. This shows only tmux-related log lines.
3. In the Ghostty window that opens, run: `tmux minus-CC new-session minus-s logtest` (note the double C — this uses DCS framing, which Ghostty will detect).
4. Watch Terminal A. You should see log lines indicating that tmux control mode was detected, that the Viewer was created, that notifications are being received, and that commands are being sent back to tmux.
5. If you see log lines mentioning the Viewer and actions, the entire pipeline from DCS detection through parsing through the Viewer is working. The Ghostty window will appear to hang because the windows action is not acted upon.
6. In a third terminal, attach to the session normally: `tmux attach minus-t logtest`. Type some commands and observe the percent-output log lines in Terminal A. Run `tmux split-window` and observe the percent-layout-change log lines.

Success criterion: You have seen Ghostty's internal tmux processing in action via log output. You can confirm that DCS detection, notification parsing, Viewer startup, command sending, and live output routing all work. You understand concretely that the pipeline works but produces no visible result because the windows action is unhandled.

Milestone 4: Instrument the code to see deeper state

Steps:

1. Open `src/termio/stream_handler.zig` and find the windows action handler (the TODO). Add a temporary log statement that prints the number of windows and, for each window, its ID and dimensions. Rebuild Ghostty.
2. Repeat the experiment from Milestone 3. Now you should see log lines showing the actual window and pane data that the Viewer discovered.
3. Open `src/terminal/tmux/viewer.zig` and find the `nextCommand` method. Add temporary log statements at each command type's handler (`list_windows`, `pane_history`, `pane_visible`, `pane_state`, `tmux_version`). Rebuild and run again.
4. Now you can see the complete command sequence: version query, window list, capture-pane for each pane (four times: primary scrollbar, primary visible, alternate scrollbar, alternate visible), and list-panes for terminal state.
5. Add a temporary log statement in the `receivedOutput` method that prints the pane ID and the length of the decoded output data. This lets you see live output routing.

Success criterion: You have added instrumentation to every major stage of the pipeline and observed the data flowing through each stage. You can describe the complete startup sequence from DCS detection through pane initialization, including every command sent and every response received, based on your log observations.

Milestone 5: Use the test framework as an exploration tool**Steps:**

1. Open `src/terminal/tmux/viewer.zig` and read the "initial flow" test (starting around line 1496). Understand the `TestStep` structure: each step has input data (simulating what tmux sends), expected actions, expected command responses, and optional check callbacks.
2. Add your own check callback to the last step of the "initial flow" test. In the callback, access the Viewer's panes map, get pane zero, and print the content of its Terminal's active screen. This shows you the actual terminal content that the Viewer captured from tmux.
3. Run the specific test: `zig build test minus-Dtest-filter equals "initial flow"` and observe the output.
4. Create your own test case that simulates a window-add notification followed by a list-windows response showing two windows. Verify that the Viewer emits a windows action with two windows and queues capture commands for the new panes.
5. Create a test case that simulates a layout-change notification adding a second pane to an existing window. Verify the Viewer creates a new Terminal instance for the new pane.

Success criterion: You have used the Viewer's test framework to observe internal state and to construct your own scenarios. You can create a `TestStep` sequence for any scenario you want to explore, verify the Viewer's behavior, and inspect the resulting state.

Approach 3: Trace-a-Feature ("Follow One Byte Through the Entire System")

When to choose this approach: Choose this when you already have some orientation (perhaps from skimming the overview documents) but need to go deep on one specific flow to understand how the layers actually connect. This approach is especially good for understanding the integration boundary where the current implementation breaks down.

Why it works: Instead of trying to understand everything at once, you pick one concrete scenario and trace it through every layer of the code. This gives you deep understanding of the specific mechanisms (function calls, data transformations, thread boundaries) that connect the layers, which is exactly what you need to implement the GUI glue.

Milestone 1: Trace the DCS detection path

Steps:

1. Open `src/terminal/dcs.zig`. Find the `tryHook` function. Read it to understand: what parameters does it check? (params equals 1000, final byte equals p). What does it do when it matches? (Creates a `control.Parser`, returns an enter notification). What is the return type?
2. Search the codebase for where `tryHook` is called. It will be called from the broader VT parser infrastructure when a DCS sequence is encountered. Note the calling function and file.
3. Now read the `put` method in the same `Handler` struct. This is called for each subsequent byte after the DCS hook. It delegates to the `tmux control Parser's put` method. Note how it returns an optional `Notification`.
4. Read the `unhook` method. This is called when the DCS String Terminator (ESC backslash) arrives. It tears down the `Parser` and returns an exit notification.
5. Write a one-paragraph summary: "When `tmux` sends ESC P 1000 p, `dcs.zig's tryHook` detects it and creates a `tmux Parser`. Subsequent bytes are routed to that `Parser` via `put`. When ESC backslash arrives, `unhook` destroys the `Parser` and emits exit."

Success criterion: You can explain, citing specific function names and return types, how `Ghostty` transitions from general VT parsing into `tmux control mode` parsing and back again.

Milestone 2: Trace one percent-output notification end-to-end

Steps:

1. Start at `control.zig`. Imagine the byte stream: percent, o, u, t, p, u, t, space, percent, 0, space, h, e, l, l, o, backslash, 0, 1, 5, backslash, 0, 1, 2, newline. Trace through the `put` method: the percent byte transitions from idle to notification state. Each subsequent byte is accumulated in the buffer. The newline triggers `parseNotification`.
2. In `parseNotification`, find the branch that handles percent-output. It uses a regex to extract the pane ID (zero) and the escaped data (hello backslash 015 backslash 012).
3. The parsed `Notification` (variant: output, with `pane_id` equals zero and `data` equals the escaped string) is returned from the `Parser`.
4. Now move to `stream_handler.zig`. The `dcsCommand` method receives this `Notification` and passes it to `viewer.next` with the notification as input.
5. In `viewer.zig`, next dispatches to `nextTmux`, which dispatches to `nextCommand` (assuming the `Viewer` is in `command_queue` state). `nextCommand` checks if the notification is an output type and calls `receivedOutput`.
6. In `receivedOutput`, find where the pane is looked up by ID in the panes hash map. The escaped data is decoded (backslash 015 becomes carriage return, backslash 012 becomes line feed). The decoded bytes are fed into the pane's `Terminal` via its VT stream parser.
7. The `Terminal` processes these bytes and updates its screen buffer. "hello" appears in the buffer followed by a carriage return and line feed.

Success criterion: You can trace one percent-output notification from raw bytes arriving at control.zig through every function call to the point where the decoded content appears in a Terminal's screen buffer. You can name every function in the chain and describe what transformation it performs.

Milestone 3: Trace the windows action to the gap

Steps:

1. In viewer.zig, find where the windows action is created. It happens in two places: after receivedListWindows completes (during initial sync or after a window-add), and after layoutChanged completes (when a layout change notification arrives). In both cases, the Viewer creates an Action with the windows variant containing the current array of Window structs.
2. The Action is returned from the next method to the caller.
3. In stream_handler.zig, find the action processing loop (around line 437). It iterates over all actions returned by viewer.next. For each action, it switches on the variant.
4. Find the windows branch. It contains a TODO comment and no implementation. This is the gap.
5. Now think about what should happen here. The stream handler knows which windows exist, their dimensions, and their pane layouts. It needs to tell the application runtime (apprt) to create corresponding native tabs and splits. Look at how the stream handler communicates with the rest of Ghostty: it has access to a termio message system and to the surface's application runtime methods.
6. Read src/termio/backend.zig and note that Kind only has exec. A tmux-backed surface would need a different kind.
7. Read src/apprt/action.zig and find the new_tab and new_split action variants. These are the operations that the stream handler would need to trigger.

Success criterion: You can explain exactly what the stream handler receives from the Viewer (an array of Window structs, each containing an ID, dimensions, and a Layout tree with pane IDs), where it currently drops this information, and what it would need to do with it (create apprt surfaces backed by a tmux backend). You can identify the specific architectural changes needed: a new Backend kind, a way to create surfaces without subprocesses, and a communication channel between the parent surface's Viewer and the child surfaces.

Milestone 4: Trace the command send path (the reverse direction)

Steps:

1. In viewer.zig, find where a command action is created. For example, after startup completes, the Viewer creates a command action with the string "display-message minus-p hash-open-brace version close-brace newline".
2. This Action is returned from next to the stream handler.
3. In stream_handler.zig, find the command action handler. It calls a write request method that queues the command string to be sent to the tmux process's standard input.
4. Trace the write request through the termio message system. The command string is packaged as a write request message, sent to the I/O thread, and written to the PTY file descriptor. tmux, running as a subprocess, receives this on its standard input.
5. tmux processes the command and sends back a percent-begin/percent-end response block, which re-enters the pipeline at step 1 of Milestone 2.

Success criterion: You can describe the bidirectional communication loop: Viewer emits command actions, stream handler sends them to tmux via the I/O system, tmux sends responses as percent-begin/percent-end blocks, the parser turns those into block_end notifications, the Viewer processes the response and may emit further actions. You can trace one complete round-trip.

Part B: Terminal and Tooling Primer

This section explains every concept, term, and tool that someone new to terminals would need for this integration. Each explanation is pitched at an engineer who is technically strong (comfortable with systems programming, data structures, state machines) but has no terminal background.

Terminal Emulator

A terminal emulator is a program that pretends to be a hardware terminal from the 1970s-1980s. Back then, you interacted with a computer through a physical device: a screen and a keyboard connected via a serial line. The computer sent characters and control codes down the serial line; the terminal displayed them. The terminal sent keystrokes back up the serial line.

A terminal emulator does the same thing in software. It creates a window on your screen, runs a shell (like zsh or bash) as a child process, and connects to that child process through a pseudoterminal. The shell thinks it is talking to a real terminal. The terminal emulator renders the output and sends keystrokes.

Ghostty is a terminal emulator. So are iTerm2, Alacritty, kitty, and Windows Terminal. Understanding that Ghostty is fundamentally a "screen plus keyboard pretending to be a 1970s terminal" is essential context for everything that follows.

PTY (Pseudoterminal)

A pseudoterminal, or PTY, is an operating system facility that creates a pair of connected file descriptors that behave like the two ends of a serial line to a hardware terminal. One end is called the master (or controller), and the other is called the slave (or secondary).

The terminal emulator holds the master end. The child process (the shell) holds the slave end. When the shell writes bytes to its stdout, those bytes appear on the master end for the terminal emulator to read and render. When the terminal emulator writes bytes to the master end (representing keystrokes), those bytes appear on the shell's stdin.

The PTY also handles some processing automatically: it translates between the terminal's raw byte stream and the kernel's line-editing mode, it manages terminal attributes (baud rate, character size, and many other settings from the serial terminal era), and it sends signals (like SIGWINCH for window size changes) to the child process.

In Ghostty, the Exec backend (src/termio/Exec.zig) creates and manages PTYs. For tmux control mode, the parent surface has a PTY (tmux runs as a child process), but tmux-backed child surfaces would not have their own PTYs.

VT Sequences (Terminal Escape Sequences)

When a program running in a terminal wants to do more than just print plain text — move the cursor, change colors, clear the screen, switch to an alternate screen buffer — it sends special byte sequences called escape sequences, VT sequences, or control sequences. They are called "VT" after the DEC VT100 terminal, which standardized many of them.

An escape sequence starts with the ESC byte (0x1B). What follows depends on the type of sequence. A CSI (Control Sequence Introducer) sequence starts with ESC followed by open-bracket and contains parameters and a final byte. For example, ESC open-bracket 31 m means "set text color to red." An OSC (Operating System Command) sequence starts with ESC close-bracket and typically sets the window title or other metadata. A DCS (Device Control String) sequence starts with ESC P and is used for more complex data transfers.

Ghostty has a full VT parser that processes these sequences. Most of this is in the terminal directory. The tmux control mode integration hooks into the DCS handling: when a DCS with parameter 1000 and final byte p arrives, it triggers the tmux control mode pipeline.

DCS (Device Control String)

A DCS is a specific type of terminal escape sequence used for complex data exchanges between a program and a terminal. It starts with ESC P (or the equivalent single byte 0x90), contains parameters, a final byte that identifies the type, and then arbitrary data terminated by a String Terminator (ST, which is ESC backslash).

tmux uses DCS to frame its control mode session. When you run tmux minus-CC, tmux sends ESC P 1000 p to signal control mode entry. Everything between that opener and the closing ESC backslash is tmux control mode protocol data. Ghostty's dcs.zig detects this specific DCS and routes the enclosed bytes to the tmux control parser.

DCS is one of several types of escape sequences that Ghostty handles. Others include CSI (cursor movement, colors, modes), OSC (title, clipboard, hyperlinks), and APC (application program commands). Each has its own handler in the terminal parsing infrastructure.

tmux

tmux is a terminal multiplexer. It runs as a server process in the background and lets multiple client sessions share a single connection. tmux manages windows (analogous to tabs) and panes (analogous to splits within a tab). Each pane contains a shell or other program.

tmux uses a server-client architecture. The server manages all the actual shell processes. A client connects to the server and provides the user interface. In normal mode, the client is a TUI (text user interface) that runs inside your terminal, drawing pane borders with box-drawing characters and rendering a status bar. In control mode, the client sends structured text messages instead of drawing a TUI.

tmux uses stable numeric identifiers with sigils: dollar sign for sessions (dollar-sign zero), at-sign for windows (at-sign one), and percent for panes (percent-sign zero). These IDs are globally unique and persistent.

tmux Control Mode Protocol

The tmux control mode protocol is the structured text format that tmux uses to communicate with smart client programs (like terminal emulators) instead of drawing its own TUI.

Every message from tmux starts with a percent sign. There are three categories of messages. First, command response blocks: when the client sends a command (like list-windows), tmux wraps the response in percent-begin and percent-end (or percent-error) lines, with the response content in between. Each guard line contains a timestamp, command number, and flags field for correlation. Second, live output: percent-output carries terminal data from a specific pane, with octal escaping for control characters. Third, async notifications: messages like percent-session-changed, percent-window-add, percent-layout-change that inform the client of state changes without being requested.

The protocol has specific escaping rules for percent-output data. Control characters (bytes below 32 decimal) are replaced with three-digit octal escapes: backslash followed by three octal digits. The backslash character itself becomes backslash-134. The client must decode these escapes before feeding the data to a VT parser.

Flow control is an optional feature where the client tells tmux to pause output for a pane if the client falls too far behind. Without flow control, a client that falls more than 300 seconds behind gets disconnected. With flow control enabled (via refresh-client minus-f pause-after equals N), tmux sends percent-pause when a pane is throttled and percent-continue when it resumes, and uses percent-extended-output instead of percent-output to include a "milliseconds behind" field.

Surface

In Ghostty's architecture, a Surface is the fundamental unit of display. It is a rectangular area where terminal content is rendered and where the user can type. In the current implementation, every Surface has a one-to-one relationship with a subprocess running in a PTY: one Surface, one shell process.

Surfaces can be arranged in the GUI as tabs (multiple Surfaces in a window, with a tab bar to switch between them) or as splits (multiple Surfaces visible simultaneously in a tiled arrangement within a single tab).

For tmux control mode, the key challenge is that tmux panes need their own Surfaces, but those Surfaces are not backed by subprocesses. They get their content from the Viewer's Terminal instances and send input through tmux's send-keys command.

Backend

In Ghostty's terminal I/O layer, a Backend is the abstraction that provides data to a Surface and accepts input from it. The only backend that exists today is the exec backend, which manages a subprocess in a PTY. The backend is responsible for starting the subprocess, reading its output, writing input to it, and cleaning up when it exits.

For tmux control mode, a new backend kind is needed. A tmux backend would not have its own subprocess or PTY. Instead, it would be connected to a specific pane in the Viewer's state, reading content from that pane's Terminal instance and sending keystrokes back through the Viewer's command queue as tmux send-keys commands.

Terminal (Ghostty's internal type)

The Terminal type in Ghostty is a complete terminal emulator state machine. It contains everything needed to represent the state of one terminal session: a screen buffer (actually two: primary and alternate), a cursor with position and attributes, mode flags (like insert mode, autowrap mode, origin mode, and several mouse tracking modes), a scroll region, tab stops, and a VT parser.

The Viewer creates one Terminal instance per tmux pane. When percent-output data arrives for a pane, the Viewer decodes the octal escapes and feeds the raw bytes into that pane's Terminal through its VT parser. The Terminal processes the bytes exactly as if they came from a local subprocess, updating its screen buffer accordingly.

For the integration, these Terminal instances are what would be rendered by tmux-backed Surfaces. The renderer does not care where a Terminal's data comes from; it just reads the screen buffer and draws pixels.

Alternate Screen Buffer

Terminals have two screen buffers: the primary screen and the alternate screen. The primary screen is where normal shell interaction happens and where scrollbar history accumulates. The alternate screen is used by full-screen applications (like vim, less, or top) that want a clean drawing surface that will not pollute the scrollbar.

When a program enters the alternate screen (via a specific escape sequence), the terminal saves the primary screen state and presents a blank screen. When the program exits the alternate screen, the terminal restores the primary screen state. This is why, after quitting vim, you see your previous shell output again.

tmux is an alternate-screen application. In normal mode, it uses the alternate screen to draw its TUI. This is relevant because the Viewer must capture both the primary and alternate screen content for each pane, since it does not know which screen is active.

Scrollbar

Scrollbar is the history of terminal output that has scrolled above the visible area. In a normal terminal, when the screen fills up and new text arrives, old text scrolls up. That old text is preserved in a scrollbar buffer and can be viewed by scrolling up.

In normal tmux usage, scrollbar is managed by tmux internally (via copy mode). In control mode, the terminal emulator (Ghostty) manages scrollbar per pane. The Viewer populates initial scrollbar for each pane by using tmux's capture-pane command with the minus-S minus flag (which means "start from the very beginning of scrollbar history").

Native scrollbar is one of the key user experience benefits of control mode: instead of using tmux's copy mode keybindings, the user can just scroll up with their mouse or trackpad as they would in any normal terminal session.

State Machine

A state machine is a programming pattern where an object has a well-defined set of states and a well-defined set of transitions between them. At any given moment, the object is in exactly one state. Events or inputs cause transitions from one state to another.

Both the tmux control Parser and the Viewer are state machines. The Parser has four states (idle, notification, block, broken). The Viewer has four states (startup_block, startup_session, command_queue, defunct). Understanding these state machines is essential to understanding the tmux integration, because the correctness of the entire feature depends on handling state transitions correctly.

In Zig, state machines are typically implemented using an enum for the state set and a switch statement for the dispatch logic. The Parser's put method switches on the current state to determine how to process each byte. The Viewer's next method switches on the current state to determine how to process each notification.

Tagged Union

A tagged union is a data type that can hold one of several variants, with a tag that identifies which variant is currently held. In Zig, this is declared with the union keyword and an associated enum for the tag.

Tagged unions are used extensively throughout the tmux integration. The Notification type is a tagged union (it can be an enter, exit, output, session_changed, etc., and the tag tells you which). The Action type is a tagged union (exit, command, or windows). The Layout's Content is a tagged union (pane, horizontal, or vertical). The Backend type is a tagged union (currently just exec, but would gain a tmux variant).

You work with tagged unions in Zig using switch statements. When you switch on a tagged union, the compiler enforces that you handle every possible variant, which prevents you from forgetting to handle a case. This is a safety feature that the tmux integration benefits from heavily: adding a new notification type to the Notification union forces you to add handling for it everywhere that union is switched on.

Oniguruma

Oniguruma is a regular expression library written in C. Ghostty's tmux control parser uses it to parse individual notification lines. For example, the percent-output notification is parsed with a regex that extracts the pane ID and the escaped data.

The build option `tmux_control_mode` is tied to Oniguruma availability. If Oniguruma is not available, the tmux module is compiled as an empty struct. In standard Ghostty builds, Oniguruma is available.

This dependency is a potential simplification target: the notification formats are simple enough that they could be parsed with basic string splitting instead of regular expressions, which would remove the Oniguruma dependency for tmux support.

Apprt (Application Runtime)

Apprt is Ghostty's abbreviation for application runtime. It is the abstraction layer that handles platform-specific GUI operations: creating windows, tabs, and splits; managing the application lifecycle; handling input events; and rendering.

Ghostty has multiple apprt implementations: one for macOS (using AppKit through Swift bindings), one for Linux (using GTK), and an embedded runtime for IDE integration. The apprt interface is defined by a set of types and methods that all implementations must provide.

For the tmux integration, the apprt layer needs to support creating Surfaces that are not backed by subprocesses. This is a fundamental change to the apprt contract, because today every Surface creation

assumes an exec backend will be set up.

Compile-time Code Generation (comptime in Zig)

Zig has a powerful compile-time evaluation feature called `comptime`. Functions and values marked `comptime` are evaluated by the compiler during compilation, not at runtime. The result is baked into the compiled binary.

The `tmux` output parser (`output.zig`) uses `comptime` extensively. The `FormatStruct` function takes an array of `Variable` values at compile time and generates a struct type at compile time. The `comptimeFormat` function generates the corresponding `tmux` format string at compile time. This means there is no runtime overhead for format string construction or struct definition: the compiler generates the exact types and strings needed.

For someone new to Zig, `comptime` can be surprising because it blurs the line between types and values. A function that returns a type is perfectly normal in Zig. The `Variable` enum is a regular runtime value, but when passed to `FormatStruct` at compile time, it produces a type that can be used to declare variables and parse data.

Arena Allocator

An arena allocator is a memory management pattern where allocations are grouped together and freed all at once. Instead of tracking each individual allocation and freeing them separately, you allocate from a contiguous region (the arena) and then free the entire region when you are done.

The Viewer uses arena allocators for layout trees. Each Window has a `layout_arena` that holds the memory for its Layout tree nodes. When a layout changes, the old arena is freed entirely and a new one is allocated for the new layout tree. This is efficient because layout trees are replaced wholesale rather than modified incrementally.

Arena allocation is common in Zig because Zig requires explicit memory management (there is no garbage collector). Arenas simplify the allocation and deallocation pattern for data structures that have a clear lifetime boundary.

Circular Buffer (CircBuf)

A circular buffer (also called a ring buffer) is a fixed-size data structure that wraps around: when you reach the end, you go back to the beginning. Items are added at the tail and removed from the head.

The Viewer uses a circular buffer for its command queue. Commands to `tmux` are added to the queue, and the Viewer sends one command at a time, waiting for the response before sending the next. The circular buffer ensures that if commands accumulate faster than responses arrive, the oldest commands can be overwritten (though in practice, the single-command-at-a-time protocol prevents this).

This completes the initial draft. It will be revised three times to improve clarity, close gaps, and fix any misleading guidance.